

LLM APIs and multimodal models

One hosted model reads text, images, audio and video. A new task needs a new prompt instead of a new dataset and a training run.

THE QUESTION

How do you get labels, entities or JSON out of a large model from a Python script, and what does each call cost?

THE INTUITION

A foundation model has already learned from internet-scale data, so you describe the task in words. Text and files go out over an API, text comes back, and tokens are what you pay for and what the limits count.

KEY TAKEAWAY

Be specific about the task and the output format, use temperature 0 for labels, ask for a schema, and check what comes back before you trust it.

key in env → client → prompt + image/audio → generate_content → text or parsed JSON → validate → cache

1 A new task is a new prompt

one pipeline per task

success Label 5,000 examples

poster objects Label 10,000 boxes

transcription collect 100 h audio

contract fields annotate 2,000 docs

4 datasets, 4 models, months

one model, one call per task

"Will this movie succeed?"

["Detect all objects.", img]

["Transcribe this.", audio]

["Extract ... as JSON.", pdf]

same model and call, new prompt

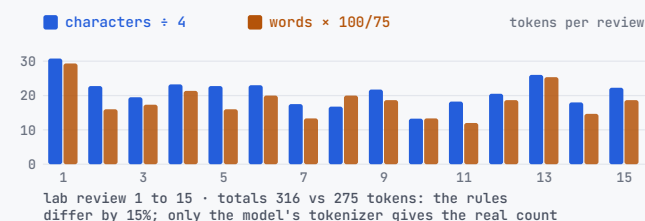
Trained on trillions of tokens, the model follows plain instructions. It still hallucinates and costs money at scale, so validation, guidelines and sampling still matter.

2 Set up once, then one call does the work

```
# pip install google-genai pillow
# export GEMINI_API_KEY='your-key-here'
from google import genai
import os
client = genai.Client(
    api_key=os.environ["GEMINI_API_KEY"])
MODEL = "gemini-2.0-flash"
q = "What is the capital of France?"
r = client.models.generate_content(
    model=MODEL, contents=q)
print(r.text) # Paris
```

Model names, free tiers and prices change often; check the provider's list before a run. Keys stay out of code (see the HTTP and APIs sheet).

3 Tokens are what you pay for and what limits count



About 4 characters or 3/4 of a word per token: 75 words is about 100 tokens. Input and output tokens are both billed. The lecture's free tier allows 15 requests a minute, which often binds before the bill does.

4 Zero-shot describes the task; few-shot shows it

Classify customer queries:

"How do I reset my password?" → Technical Support

"I was charged twice" → Billing

"What are your hours?" → General Inquiry

Query: "My app keeps crashing"

Category:

instruction

examples

new input

cue

↓ model continues: Technical Support

Zero-shot: name the allowed labels, add "Reply with ONLY the label". **Few-shot:** add worked examples; the model copies the pattern and no weights change. **Chain of thought:** ask for the steps before the answer.

5 Chat APIs take role-tagged messages

OpenRouter, the lab's second provider, speaks the OpenAI format.

```
import openai
client = openai.OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"])
r = client.chat.completions.create(
    model=FREE_MODELS["llama-3.1-8b"],
    temperature=0, messages=[
        {"role": "system", "content": RULES},
        {"role": "user", "content": review}])
r.choices[0].message.content
```

system holds standing rules (Gemini: system_instruction); a past user/assistant pair acts as a shot. FREE_MODELS is the lab's dict of free model ids.

6 Temperature sharpens or flattens the odds

next word after "The movie was", illustrative logits

word	logit	T = 0	T = 0.2	T = 0.7	T = 1.0	T = 1.5
great	+3.0	1.00	0.98	0.71	0.60	0.48
good	+2.2	0.00	0.02	0.23	0.27	0.28
okay	+1.0	0.00	0.00	0.04	0.08	0.13
boring	+0.3	0.00	0.00	0.02	0.04	0.08
purple	-1.5	0.00	0.00	0.00	0.00	0.01

$p_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$; $T = 0$ means always take the top word

{"temperature": 0} for facts, code and labels; 0.7 for general tasks; 1.0 or more for creative writing. At 0 the output repeats closely, but providers do not promise identical text.

Ask for a schema, then check the answer

Typed JSON from a Pydantic model, images and audio in the call, and what 1,000 calls cost.

7 A response schema returns typed JSON

schema in config (the lecture)



JSON asked for in the prompt (the lab)



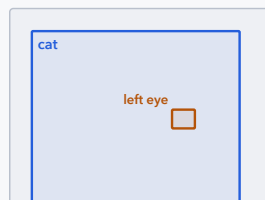
a schema fixes the shape of the answer, not its truth: still check values

```

from pydantic import BaseModel
class Person(BaseModel):
    name: str
    age: int
r = client.models.generate_content(
    model=MODEL, contents=
    "Extract: Sarah is 34 years old",
    config={"response_mime_type":
            "application/json",
            "response_schema": Person})
r.parsed.age # 34
  
```

The lecture calls this guaranteed JSON. The shape is enforced, but values can be invented or cut off at `max_output_tokens`, so validate as on the data validation sheet.

8 Images go in the contents list



```

box_2d = [116, 85, 1000, 885]
order [y0, x0, y1, x1], 0 to 1000
x_px = x / 1000 * width
y_px = y / 1000 * height
on an 800 x 600 image (assumed)
cat: x 68 to 708, y 70 to 600 px
  
```

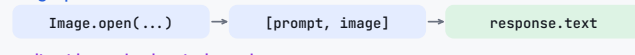
```

from PIL import Image
image = Image.open("photo.jpg")
prompt = """Detect objects. Return JSON:
[{"label": "cat",
  "box_2d": [y0, x0, y1, x1]}]"""
r = client.models.generate_content(
    model=MODEL, contents=[prompt, image])
  
```

Receipts ("Extract store name, items, total as JSON.") and handwritten maths ("Solve step by step.") work the same way.

9 Audio and video are uploaded first

image: pass it inline



audio, video: upload, wait, then ask



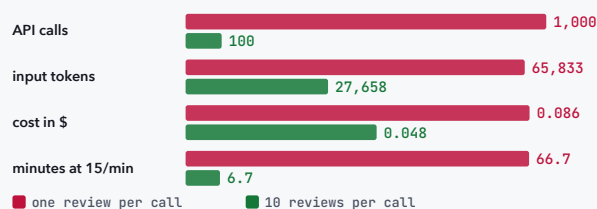
```

f = client.files.upload(file="scene.mp4")
while f.state.name == "PROCESSING":
    time.sleep(5)
    f = client.files.get(name=f.name)
r = client.models.generate_content(
    model=MODEL, contents=[
        "What happens in this video?", f])
  
```

Audio: upload speech.mp3, ask "Transcribe this."

10 Batching saves requests more than money

1,000 reviews, illustrative (prices and sizes below)



Lab prompts sized by the 4-character rule, at the lab's \$0.001 and \$0.002 per 1K input and output tokens. Check 10 labels came back for 10 reviews.

11 Cache repeats and pace the calls

```

cache = {}
def ask(prompt, t=0):
    key = (MODEL, prompt, t)
    if key not in cache:
        r = client.models.generate_content(
            model=MODEL, contents=prompt,
            config={"temperature": t})
        cache[key] = r.text
        time.sleep(4) # 15 calls/min
    return cache[key]
  
```

Save it to disk; use "flash" before "pro"; back off on 429 (HTTP and APIs sheet).

12 Pause and think

Few-shot says Mixed; zero-shot never does. Why? Only the few-shot prompt offers Mixed.
10,000 single calls at 15 a minute? About 11 hours; 10 per call, 67 minutes.

Why "Reply with ONLY the label"? Else "The sentiment is Positive." hits your parser.
Schema output parsed. Is it correct? Only its shape; spot-check a sample.

Box [519, 625, 615, 713] on 800 x 600? x 500 to 570, y 311 to 369 px.

Other uses? Paraphrases at temperature 0.7; fixing "two thousand ten" to 2010.

key in env · exact task and format · temperature 0 for labels · schema, then validate · batch, cache, pace

Source: lecture "Foundation models in practice" and its lab "LLM APIs and prompt engineering", nipunbatra.github.io/stt-ai-teaching. Code uses the google-genai SDK with gemini-2.0-flash (the lab uses gemini-2.0-flash-exp) and OpenRouter's free models as taught; model names, free tiers and prices change. Token counts, logits, the 800 x 600 image and the cost comparison are illustrative, computed from the lecture's rules of thumb and the lab's assumed prices. LLM labelling in depth is on the labelling-less sheet.